

PNGres

A Primate-Addressable Persistence Layer

TECHNICAL SPECIFICATION · VERSION 0.2 · 2026

Abstract. PNGres is a local-first data store in which every stored byte is represented as a low-order bit of a 650×800 RGBA image of a monkey. Arbitrary binary inputs are sharded across a sequence of PNG files that are

visually indistinguishable from the cover image. Payload data is embedded in the least-significant bits of the pixel stream; per-file structural metadata is carried in private PNG ancillary chunks. The system supports byte-accurate random access, which enables HTTP range requests and, by extension, live video playback directly from the resulting herd. A Server-Sent Events channel surfaces every interaction (including peer IPs) as a live feed, and a lightweight P2P pull protocol allows independent instances to exchange monkes over plain HTTP. This document specifies the on-disk format, the HTTP API surface, the event model, the P2P protocol, and observed performance characteristics.

CHAPTER 0

Contents

Motivation 2
Architecture Overview 3
  Request Flow: Range-Streamed MP4 3
On-Disk Format 4
  Shard Structure 4
  `mkHd` — Fixed 64-Byte Header 4
  `mkMf` — Manifest JSON 5
  `mkCk` — Per-Shard Integrity Hash 5
  LSB Embedding Layout 5
  Capacity 5
  File-to-Herd Expansion 6
HTTP API 7
  Endpoint Reference 7
  Range Request Semantics 7
  Error Taxonomy 8
Event Bus and Live Observability 9
  Event Kinds 9
  SSE Wire Format 9
  Snapshot and Peer Endpoints 10
P2P Protocol 11
  Pull Flow 11
  Caching and Fan-Out 11
  Asynchronous Transfer 11
CLI Reference 12
  Subcommands 12
  Exit Codes 12
Performance 14
  Throughput (2-bit LSB) 14
  First-Byte Latency 14
  Memory Bounds 14
Operator's Handbook 15
  Environment Variables 15
  Deployment Modes 15
  Backup 15
Testing Summary 16
Limitations and Future Work 17
Glossary 18
References 19

CHAPTER 0

Motivation

The author was asked, verbatim, for "a database tool that ONLY stores data for any file type or any type of data in headers and data of the monke.png file thats in this dir." The request further stipulated that the resulting system be funny, fully functional, streaming-capable, and scalable; that there be a CLI and a web UI; that the CLI be able to pack monkes into zip files; and that peers be able to stream monkes to each other. PNGres is the response.

We frame the monke as a *content-addressable storage primitive*. Each shard is a self-describing unit of approximately 508 KiB of payload (at the default 2-bits-per-channel density), carrying a complete copy of the file-level manifest in a private ancillary chunk. This allows the system to tolerate a surprising range of operational hazards — partial loss, metadata stripping, filesystem reordering, re-encoding by image viewers — while still presenting the user with what they asked for, which is a directory of monkes.

> **DESIGN INVARIANT**
> The filesystem is the database. There is no SQLite, no Postgres, no Redis, and no on-disk index. `data/<file_id>/monke_*.png` is authoritative. A scan of that directory, reading only the `mkMf` chunk of each first shard, fully reconstructs server state.

CHAPTER 0

Architecture Overview

The implementation is a Rust Cargo workspace of three crates plus a static frontend bundle:

Crate	Responsibility
`pngres-core`	Pure encoding/decoding library. PNG chunk I/O, LSB embed/extract, manifest (de)serialization, streaming random-access reader. No async, no HTTP, no opinions about deployment. Usable as a standalone Rust crate.
`pngres-server`	HTTP frontend on `axum 0.7` . Filesystem-backed in-memory index, shard storage under `data/<file_id>/` , embedded static assets via `rust-embed` , `tokio::sync::broadcast` event bus with a 500-event ring buffer powering the SSE live feed.
`pngres-cli`	Command-line interface on `clap 4` . Directory and zip I/O. Feature-gated `serve` subcommand. `remote` subcommand speaks to a running server (P2P pull/list/discover/mirror).
`web/dist/`	HTML + CSS + vanilla JavaScript frontend. No Node.js toolchain; approximately 900 lines of code total. Renders the LSB amplification visualization entirely client-side using a `<canvas>` .

Request Flow: Range-Streamed MP4

```
[browser] --GET /api/files/:id/stream  Range: bytes=A-B⟶
[server]  --index.get_by_str(id) -- in-memory HashMap lookup⟶
[server]  --compute first = A / per_monke_payload_bytes
                  last  = B / per_monke_payload_bytes⟶
[server]  --open only shards [first..=last] on disk⟶
[core]    --PNG decode + LSB extract for each shard
           (spawn_blocking to yield the tokio worker)⟶
[server]  --Body::from_stream(Stream<Result<Bytes>>)⟶
[browser] ⟵206 Partial Content, Content-Range: A-B/total--
[server]  --emit Event::Stream { peer_ip, range, slice_len }⟶
[ui]      ⟵SSE /api/events pushes the event to every subscriber--
```

Every arrow here is real and minimal. There is no queue, no worker pool (beyond the default `tokio` scheduler), no caching layer. The entire round-trip from browser scrub to monke re-decode to byte delivery lives inside one `axum` handler.

CHAPTER 0

On-Disk Format

Shard Structure

A shard is a valid, viewer-compatible PNG file. Chunks appear in the following order:

Chunk	Type	Purpose
`IHDR`	standard	650×800, 8-bit RGBA, non-interlaced.
`mkHd`	ancillary / private	PNGres fixed 64-byte header.
`mkMf`	ancillary / private	Full file manifest, JSON-encoded. Duplicated in every shard.
`IDAT`	standard	Pixel data with payload LSB-embedded.
`mkCk`	ancillary / private	32-byte BLAKE3 of this shard's payload.
`IEND`	standard	PNG terminator.

All `mk*` chunks follow the PNG convention for *ancillary, private, safe-to-copy* chunks: the first and second letters are lowercase (ancillary / private), the third is uppercase (reserved), the fourth is lowercase (safe for viewers to copy across re-saves).

`mkHd` — Fixed 64-Byte Header

Off	Size	Field	Notes
0	8	`magic`	ASCII `"PNGRES01"` .
8	1	`format_version`	`0x01` for this spec.
9	1	`lsb_bits`	1, 2, or 4.
10	2	`flags`	bit 0 reserved for payload compression.
12	16	`file_id`	UUIDv7 raw bytes (time-sortable).
28	4	`chunk_index`	u32 big-endian, 0-based.
32	4	`total_chunks`	u32 big-endian.
36	8	`byte_offset`	u64 BE, payload's offset in the original file.
44	4	`payload_length`	u32 BE, actual bytes in this shard.
48	8	`total_file_size`	u64 BE.
56	4	`header_crc32`	CRC32(bytes 0..52).
60	4	`reserved`	Zero.

The `mkHd` appears *twice* in every shard: once as a PNG ancillary chunk (fast, survives most re-encodes), and once mirrored into the first 64 bytes of the LSB-embedded payload stream (slower, survives chunk stripping). The CRC32 is over the mutable portion so tampering is detected either way.

`mkMf` — Manifest JSON

Serialized form of the `pngres_core::Manifest` struct, `serde_json` -encoded:

```
{
  "file_id":           "019dbe71-4699-72f0-90df-f6d37d8fe0cd",
  "filename":          "movie.mp4",
  "mime":              "video/mp4",
  "total_size":        106290,
  "num_monkes":        1,
  "lsb_bits":          2,
  "payload_per_monke": 519904,
  "file_hash":         [ 32 bytes of BLAKE3 ],
  "shard_hashes":      [[ 32 bytes ], ...],
  "created_at":        1777016587,
  "format_version":    1
}
```

`mkCk` — Per-Shard Integrity Hash

32 bytes. BLAKE3 of the shard's payload bytes (not of the whole PNG).

LSB Embedding Layout

The payload is embedded into the low `lsb_bits` bits of each pixel byte, in document order (R, G, B, A, R, G, B, A, …). The full LSB stream carries:

```
| 64 B  mkHd mirror | payload_length bytes of data | 32 B  BLAKE3 footer |
```

The mirror is redundant with the ancillary `mkHd` chunk, by design. If a PNG re-saver strips the `mk*` chunks — some do, despite the safe-to-copy bit — the decoder falls back to LSB extraction. We try 2, 1, and 4 bit depths in order, and the first valid magic + CRC wins. The one scenario that defeats both paths is a lossy re-encoding (JPEG conversion, heavy compression with dithering), which scrambles the low bits.

Capacity

A single 650 × 800 × 4 = 2,080,000-channel-byte monke carries:

LSB bits	Raw capacity	User payload	Visible distortion?
1	260,000 B	253.8 KiB	invisible
2	**520,000 B**	**507.7 KiB**	**almost invisible (default)**
4	1,040,000 B	1,015.5 KiB	mild banding in smooth regions

Subtracting the 96 bytes of LSB overhead (64 B header mirror + 32 B payload hash) leaves the user payload number quoted.

File-to-Herd Expansion

For an input file of size S bytes at bit depth b, the shard count is:

$$N = \left\lceil \frac{S}{P_b} \right\rceil$$

where P_b is the per-monke user payload at bit depth b. Each shard occupies about 700 KiB on disk after PNG compression at `Compression::Fast` ; the LSB churn is high-entropy and compresses poorly, yielding a storage blow-up factor of roughly **1.4×** relative to the source.

CHAPTER 0

HTTP API

All routes return JSON unless otherwise noted. Errors return `{ "error": { "code": <string>, "message": <string>, "hint": <string> } }` where `hint` carries an informative, deliberately unserious remark.

Endpoint Reference

Method	Path	Purpose
`GET`	`/healthz`	Plain text `"ok"` .
`GET`	`/api/stats`	`{ total_files, total_bytes, total_monkes, oldest, newest }` .
`POST`	`/api/upload`	`multipart/form-data` ; field `file` . Returns `{ id, name, mime, size, num_monkes, uploaded_from }` .
`GET`	`/api/files?limit&offset`	Paginated list, newest-first.
`GET`	`/api/files/:id`	Summary + full `Manifest` .
`GET`	`/api/files/:id/download`	Full body, `Content-Disposition: attachment` .
`GET`	`/api/files/:id/stream`	Range-aware. 206 on Range, 200 otherwise. `Accept-Ranges: bytes` .
`GET`	`/api/files/:id/monkes`	P2P: index of raw shard URLs. Does not decode.
`GET`	`/api/files/:id/monkes/:n`	P2P: the n-th raw shard PNG. `Cache-Control: immutable` .
`DELETE`	`/api/files/:id`	Delete file + shards (hard-removed from disk).
`GET`	`/api/events`	Server-Sent Events live feed (see Chapter 5).
`GET`	`/api/activity`	JSON snapshot of the recent event ring buffer.
`GET`	`/api/peers`	Unique peer IPs + event counts + first/last seen.
`GET`	`/` , `/static/{*path}`	Embedded static frontend.

Range Request Semantics

Given `Range: bytes=A-B` :

1. The server validates the range against `manifest.total_size` via `http_range_header::validate` . Out-of-bounds ranges yield `416` .
2. Suffix ranges (`bytes=-N`) and open-ended ranges are capped at 8 MiB so that browsers issue follow-up Range requests rather than streaming the whole file on the first GET. This keeps peak memory flat regardless of file size.

3. The covering shard window is computed as `first = A / per` , `last = B / per` , where `per = manifest.payload_per_monke` . Only those shards are opened.
4. Multi-range (`Range: bytes=0-99,200-299`) is rejected with `416` . Common browsers do not send multi-range for `<video>` ; we can add `multipart/byteranges` later if needed.

Error Taxonomy

HTTP	`error.code`	When
400	`bad_request`	Malformed multipart, missing `file` field.
404	`not_found`	Unknown `file_id` , out-of-range monke index.
413	`too_large`	Upload exceeds `PNGRES_MAX_UPLOAD_BYTES` .
416	`bad_range`	Range unsatisfiable or multi-range.
500	`core_error`	`pngres-core` returned an `Error` (corrupt shard, hash mismatch, missing shard).
500	`io_error`	Filesystem trouble — permissions, disk full, missing shard file.
500	`internal`	Anything else — JSON parse, task join, tokio weirdness.

CHAPTER 0

Event Bus and Live Observability

Every handler that succeeds emits a typed `Event` onto an internal `tokio::sync::broadcast` channel. The bus also maintains a bounded ring buffer (default 500 entries) so a fresh SSE client can receive recent history before it starts streaming live.

Event Kinds

Kind	Emitted when
`index_ready`	Server has finished scanning `data/` at startup.
`upload`	A file has been accepted, encoded, and written.
`download`	A full-file download has been served.
`stream`	A `/stream` request has been answered (with or without a Range).
`shard_served`	A single raw shard has been served via `/monkes/:n` .
`index_listed`	A `/monkes` index has been returned — typically the opening move of a P2P pull.
`delete`	A file has been removed.

Each non-startup event carries:

Field	Semantics
`ts`	Unix seconds.
`peer`	Peer IP (via `axum::extract::ConnectInfo`). IPv4 or IPv6. `null` on startup events.
`id`	File identifier (UUIDv7 string).
`name`	Original filename.
`chunk_index` , `total`	Populated only for `shard_served` .
`range_start` , `range_end_inclusive` , `slice_len`	Populated only for `stream` .
`num_monkes`	Populated for `upload` and `index_listed` .
`size` or `bytes`	Byte count, per event kind.

SSE Wire Format

`GET /api/events` returns `text/event-stream` . Each frame:

```
data: {"kind":"shard_served","ts":1777019574,"peer":"203.0.113.42",
"id":"019d...","name":"clip.mp4","chunk_index":3,"total":7,"bytes":833235}
```

Clients using `EventSource` receive these automatically. The connection is kept alive with a 15-second heartbeat comment. On reconnect, the client first receives the current snapshot, then switches to live.

Snapshot and Peer Endpoints

`GET /api/activity` returns a JSON array of the ring buffer (up to 500 most-recent events). `GET /api/peers` returns deduplicated peers aggregated from the same buffer, each with `first_seen` , `last_seen` , and `events` (count).

These are intended for clients that cannot or will not speak SSE — mobile browsers in power-save mode, `curl` -based dashboards, or the occasional engineer with a grudge.

CHAPTER 0

P2P Protocol

PNGres instances can pull monkes from each other using only HTTP `GET` . There is no gossip, no signalling, no discovery. You tell your `pngres-cli` about a remote base URL; it fetches the shards; your local `pngres-core` decodes them. The protocol is deliberately dumb.

Pull Flow

Given a remote `BASE` and a known `file_id` , the client performs:

1. `GET BASE/api/files/:id` — returns a `FileDetail` including the complete `Manifest` with `file_hash` , per-shard hashes, and `num_monkes` . This is everything the client needs to verify the rest.
2. `GET BASE/api/files/:id/monkes` — returns an ordered `MonkeIndex` of URLs. The client could have derived these from the manifest alone; the endpoint is a convenience.
3. For `n` in `0 .. num_monkes` : `GET BASE/api/files/:id/monkes/:n` . Each response is a valid PNG with `Content-Type: image/png` . The client calls `pngres_core::decode_shard_bytes` locally, which:
   1. Verifies the `mkHd` CRC32.
   2. Verifies `shard.header.file_id` matches the expected one.
   3. Verifies `shard.header.chunk_index` matches `n` .
   4. Verifies the LSB-embedded footer BLAKE3 matches `hash_bytes(payload)` .
4. The client concatenates payloads in `chunk_index` order and verifies the whole-file BLAKE3 against `manifest.file_hash` .

At no point does the client trust the remote server's interpretation of bytes. The only trust is "this remote served bytes with my requested shape." Lying produces a crashing decode.

Caching and Fan-Out

Each `/monkes/:n` response sets `Cache-Control: public, max-age=31536000, immutable` . Shards are content-addressed by their embedded BLAKE3, so any HTTP cache (CDN, Varnish, `curl --cache`) is free to hold them forever without staleness concerns. This makes PNGres trivially compatible with BitTorrent-style parallel fan-out: point N clients at the same shard URL, cache the result anywhere, the bytes do not lie.

Asynchronous Transfer

The `pngres encode --zip` + `pngres decode <.zip>` flow is the same protocol, reified as files. The zip is a content-addressed bundle of monkes. It can cross any medium — email, USB stick, Signal, carrier pigeon. A `decode` at the other end performs the same verification chain as the live pull.

CHAPTER 0

CLI Reference

The CLI wraps `pngres-core` directly. For any `encode` invocation, the resulting herd is recoverable through `decode` without any server.

Subcommands

Invocation	Behaviour
`pngres encode <FILE> --out <DIR>`	Write one PNG per shard into `DIR`, plus a sidecar `manifest.json`.
`pngres encode <FILE> --zip <PATH>`	Write a single `.zip` of PNG shards + manifest.
`pngres encode ... --bits 1|2|4`	Choose bit depth (default 2).
`pngres encode ... --force`	Overwrite an existing destination.
`pngres decode <SOURCE> --out <FILE>`	Reassemble; `<SOURCE>` may be a directory or a zip.
`pngres decode ... --no-verify`	Skip the whole-file BLAKE3 check (per-shard hash is still verified).
`pngres inspect <MONKE>`	Human-readable metadata.
`pngres inspect <MONKE> --json`	Machine-readable.
`pngres capacity [--bits N]`	Capacity-per-monke table.
`pngres serve --data-dir <DIR>`	Launch the server (requires `--features serve`, default).
`pngres remote list <URL>`	List files on a remote instance.
`pngres remote discover <URL> <ID>`	Print a remote file's monke index.
`pngres remote pull <URL> <ID> --out <FILE>`	Stream a decoded file from a remote (BLAKE3-verified).
`pngres remote pull-monkes <URL> <ID> --out <DIR>`	Pull raw shard PNGs for local decode.
`pngres remote mirror <URL> --out <DIR>`	Pull every file from a remote.

Exit Codes

Code	Meaning	Typical Trigger
0	success	no trigger
1	io error	permissions, disk, missing file
2	usage	clap-level CLI mistake

3	corruption	chunk CRC mismatch, mangled PNG, tampered pixels
4	missing shard	a monke in the sequence is absent
5	hash mismatch	shard or whole-file BLAKE3 mismatch on decode
6	destination exists	output path present, `--force` not supplied

CHAPTER 0

Performance

Measured on Apple silicon (arm64, 8-core M-series, `rustc 1.95` , release profile with `lto = "thin"`), via `cargo bench -p pngres-core --bench throughput` . Bench harness: `criterion 0.5` , 10 samples per case.

Throughput (2-bit LSB)

Operation	64 KiB	1 MiB	10 MiB
Encode (parallel, rayon)	60 MiB/s	182 MiB/s	473 MiB/s
Decode (sequential)	24 MiB/s	64 MiB/s	72 MiB/s

Encode scales near-linearly with core count because every shard is independent: clone the `Arc<[u8]>` cover, twiddle LSBs, re-encode PNG at `Compression::Fast` . The bottleneck is the `miniz_oxide` deflate encoder; `mtpng` would make this faster if latency mattered. It does not.

Decode is deliberately single-threaded so that HTTP range consumers see strictly ordered bytes without reorder buffers. This is a latency-over- throughput tradeoff; a parallel decoder is feasible.

First-Byte Latency

For a cold `/stream` request with `Range: bytes=X-Y` :

1. One in-memory `HashMap` lookup (100 ns).
2. One open + read of a single monke PNG file (ms on local SSD).
3. `miniz_oxide` inflate of one IDAT (10 ms for a 700 KiB monke).
4. LSB extraction of `Y - X + 1` bytes (1 ms).

End-to-end under 15 ms in practice, dominated by PNG inflate. The `tokio::task::spawn_blocking` hop costs a thread pool dispatch; we accept it for correctness under load.

Memory Bounds

- Encoder: roughly `num_cores × (2 MB cover + 0.5 MB payload + ~1 MB scratch)` . About 40 MB on an 8-core machine regardless of input size.
- Upload path (v1): buffers the entire multipart field in memory before encoding. A 2 GiB upload on a 4 GiB machine is unwise.
- Decoder: one shard in flight per logical stream. Yields chunks of `payload_per_monke` or less.

CHAPTER 0

Operator's Handbook

Environment Variables

Variable	Default	Meaning
`PNGRES_HOST`	`127.0.0.1`	Bind address. Set to `0.0.0.0` to accept LAN/remote.
`PNGRES_PORT`	`6969`	TCP port.
`PNGRES_DATA_DIR`	`./herd`	Where shards are stored.
`PNGRES_MAX_UPLOAD_BYTES`	`2147483648`	Upload cap (2 GiB default).
`RUST_LOG`	`pngres_server=info,tower_http=info`	tracing filter.

Deployment Modes

- **Laptop / localhost**: `just serve` . Browser at `http://127.0.0.1:6969` .
- **LAN**: `PNGRES_HOST=0.0.0.0 just serve` ; share the machine's LAN IP. macOS firewall will prompt on first connection.
- **Public via tunnel**: run an unprivileged tunnel in front (we tested `cloudflared tunnel --url http://127.0.0.1:6969`). The ephemeral URL propagates through DMs trivially.
- **Behind a reverse proxy**: terminate TLS at the proxy. Set `X-Forwarded-For` if you want accurate peer IPs; otherwise the bus will report the proxy's address. (A future release may honour `X-Forwarded-For` natively; v0.2 does not.)

Backup

Back up `data/` . That is the entire store. The in-memory index is rebuilt on next start. No other state exists.

CHAPTER 0

Testing Summary

Green at time of writing:

Count	Coverage
21	Core unit tests (`capacity` , `chunk` , `cover` , `encode` , `hashing` , `lsb` , `manifest`).
14	Real-fixture roundtrips (txt/rs/json/pdf/png/jpg/mp3/mp4/empty/1-byte, MP4 at 1/2/4 LSB, 25 random byte-ranges into MP4).
11	Synthetic-payload roundtrips incl. boundary and cross-boundary cases.
3	Random-access tests (crossing, zero-length, past-EOF, missing-shard).
3	Property tests (`proptest`) — arbitrary `Vec<u8>` roundtrips at each bit depth.
1	`mkHd` strip → LSB-mirror fallback decodes correctly.
5	CLI happy-path via `assert_cmd` incl. dir+zip roundtrip and the exit-code-6 dest-exists case.
3	P2P API: full MP4 flow, multi-fixture stats, pull-then-redecode preserves bytes across 5 file types.
2	Event bus + peer capture: every event kind is emitted with a loopback peer IP; `uploaded_from` propagates through the API.

Total: 63 tests across 10 test binaries.

End-to-end streaming is verified against a real H.264 MP4 fixture: the test harness boots the server in-process on an ephemeral port, uploads the fixture, issues HTTP Range requests, diffs the response bytes against the original slice via `cmp` , and additionally validates the reconstructed whole-file MP4 via `ffprobe` to confirm the stream is structurally intact (codec, dimensions, duration all match the source).

CHAPTER 0

Limitations and Future Work

- **No encryption.** LSB steganography is obfuscation, not secrecy. Encrypt inputs before handing them to PNGres if confidentiality matters.
- **No deduplication.** Identical uploads produce disjoint herds. Trivial to add on the back of a content-hash index (`HashMap<[u8;32], Uuid>`).
- **In-memory upload buffer.** The v1 upload path reads the whole multipart field into memory before encoding. A streaming variant is feasible given the shard-parallel encoder.
- **In-memory index scales with herd count.** Startup scans every file dir once to rebuild the manifest cache. For 100,000 files this is measurable; a persisted sidecar or a content-addressed store would solve it.
- **In-memory event log.** The 500-event ring buffer does not survive restarts. A `jsonl` tee on disk is one `tokio::spawn` away.
- **Single cover image.** `pngres-core` embeds `monke.png` via `include_bytes!` and assumes 650×800 RGBA. A cover registry with per-shard cover identifiers is architecturally straightforward.
- **No erasure coding.** Lose a monke, lose the file. Reed–Solomon `(k, n)` coding would make any `k` -of- `n` subset sufficient for recovery.
- **Single-writer.** The index is an `RwLock<HashMap>` . Concurrent uploads serialize at the insert point. At laptop scale this is fine.
- **Browser LSB amplifier** does CPU-side XOR+shift per pixel in a synchronous `for` loop. For the fixed 650×800 cover this is fast; for larger covers it would belong in a `Worker` .

CHAPTER 0

Glossary

monke The atomic unit of PNGres storage. One 650×800 RGBA PNG carrying up to 508 KiB of user payload at the default bit depth.

herd A collection of monkes forming the complete encoding of one file. Formally the set $\{m_0, ..., m_{n-1}\}$ where n is the shard count.

LSB (Least-Significant Bit) The lowest-order bit of a byte. At `lsb_bits = 2` we use the two lowest bits of every channel byte.

ancillary chunk A PNG chunk whose first letter is lowercase, signalling to PNG decoders that the chunk may be ignored without loss of image information. PNGres defines `mkHd` , `mkMf` , and `mkCk` .

manifest The file-level metadata record — filename, MIME, total size, shard count, bit depth, BLAKE3 hash of the whole file, per-shard hashes.

chunk_index The 0-based position of a shard within its herd. Equivalent to the shard's payload byte offset divided by the per-monke payload size.

payload_per_monke The number of user bytes that fit into a single shard after subtracting the 96-byte LSB overhead (64 B header mirror + 32 B payload hash) at a given bit depth.

uploaded_from The IPv4/IPv6 address that performed the upload, as observed by the server. Captured from the TCP peer address via `axum::extract::ConnectInfo` . Lost on server restart.

CHAPTER 0

References

1. **PNG (Portable Network Graphics) Specification**, W3C Recommendation, Third Edition, 2003; IETF RFC 2083. https://www.w3.org/TR/png/
2. **BLAKE3 Cryptographic Hash**, Aumasson, Neves, Wilcox-O'Hearn, Winnerlein, 2020. https://github.com/BLAKE3-team/BLAKE3-specs
3. **UUID Version 7**, IETF RFC 9562. Time-sortable UUID format. https://datatracker.ietf.org/doc/rfc9562/
4. **HTTP Semantics — Range Requests**, IETF RFC 9110 §14. https://datatracker.ietf.org/doc/html/rfc9110
5. **Server-Sent Events**, WHATWG HTML Living Standard §9.2. https://html.spec.whatwg.org/multipage/server-sent-events.html
6. **axum** — web framework, tokio-rs. https://docs.rs/axum/
7. **rayon** — data-parallel iterators for Rust. https://docs.rs/rayon/
8. **Typst** — this document's typesetter. https://typst.app/